

Bài 1: Hãy sử dụng activation sigmoid và xem kết quả.

```
✓ [68] y_pred1 = model1.predict(X_test_resaped)
```

10s

↔ 313/313 ————— 5s 16ms/step

```
✓ [69] y_pred1 = np.argmax(y_pred1, axis = -1)
```

0s

```
✓ [70] y_pred1[0]
```

0s

↔ np.int64(7)

```
✓ [71] y_test[0]
```

0s

↔ np.uint8(7)

```
✓ [72] accuracy_score(y_test, y_pred1)*100
```

0s

↔ 87.96000000000001

Bài 2: Hãy sử dụng activation relu cho lớp thứ 1 và activation sigmoid cho lớp đầu ra và xem kết quả.

```
✓ [75] y_pred2 = model2.predict(X_test_resaped)
```

2s

↔ 313/313 ————— 2s 7ms/step

```
✓ [76] y_pred2[0]
```

0s

↔ array([0.00000000e+00, 1.12587255e-27, 7.44499356e-22, 3.32383061e-16,
3.05921286e-35, 1.87778219e-25, 0.00000000e+00, 1.00000000e+00,
7.64594010e-17, 1.43180904e-25], dtype=float32)

```
✓ [77] y_pred2 = np.argmax(y_pred2, axis = -1)
```

0s

```
✓ [78] accuracy_score(y_test, y_pred2)*100
```

0s

↔ 97.25

Bài 3: Normalizing giá trị cho data như sau:

```
✓ [87] y_pred3 = model2.predict(X_test_new_resaped)
5s
```

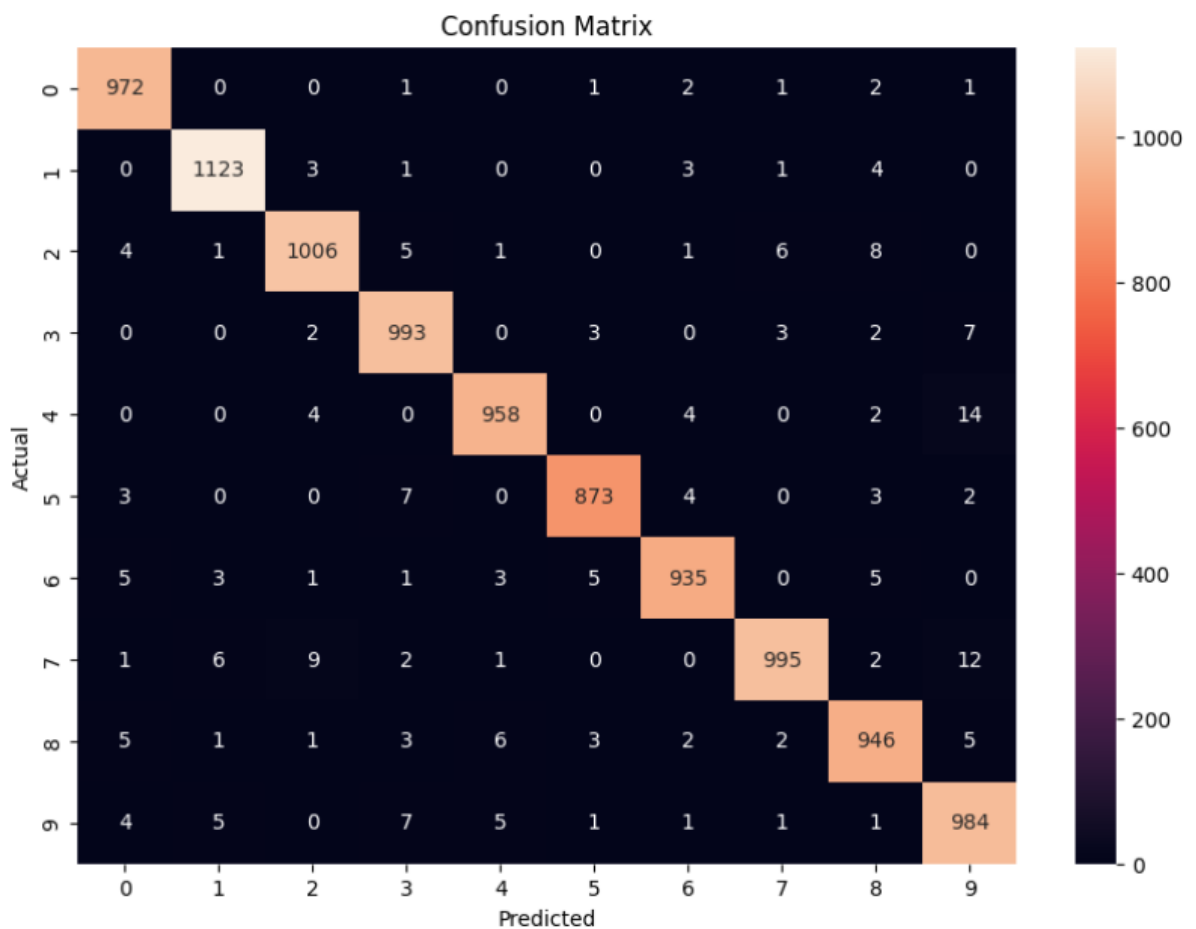
313/313 ————— 5s 16ms/step

```
✓ [88] y_pred3 = np.argmax(y_pred3, axis = -1)
0s
```

```
✓ [89] accuracy_score(y_test, y_pred3)*100
0s
```

97.85000000000001

Bài 4: In ra ma trận nhầm lẫn của mô hình ở bài 3 và nhận xét.



Nhận xét:

Độ chính xác cao: Các giá trị lớn tập trung trên đường chéo chính, cho thấy mô hình có độ chính xác cao trong việc phân loại các lớp.

Một số nhầm lẫn nhỏ: Có một số lỗi dự đoán xuất hiện nhưng với số lượng nhỏ, ví dụ:

- Lớp 2 bị nhầm thành lớp 8 (8 lần).
- Lớp 4 bị nhầm thành lớp 9 (14 lần).
- Lớp 7 bị nhầm thành lớp 2 (9 lần).

Dữ liệu cân bằng: Số lượng mẫu trong mỗi lớp khá đồng đều, không có lớp nào quá ít hoặc quá nhiều.

Mô hình hoạt động tốt trên hầu hết các lớp: Hầu hết các lỗi phân loại có số lượng thấp, chứng tỏ mô hình hoạt động tốt trên nhiều lớp.

Bài 5: *Hãy xây dựng model đơn giản gồm 2 lớp cho bộ dữ liệu **small CIFAR10**.

Xây dựng với 2 lớp cơ bản:

Lớp 1: Conv -> Pooling

Lớp 2: Fully Connected (Output)

```
model5_1 = Sequential([
    # Lớp 1: Conv -> Pooling
    Conv2D(filters=32, kernel_size=(4,4), activation='relu', input_shape=(32,32,3)),
    MaxPool2D(pool_size=(2,2)),

    # Lớp 2: Fully Connected (Output)
    Flatten(),
    Dense(10, activation='softmax') # 10 lớp đầu ra
])
```

```
model5_1.compile(loss='categorical_crossentropy',
                  optimizer='adam',
                  metrics=['accuracy'])
```

```
model5_1.summary()
```

```
✓ [25] # Đánh giá độ chính xác
1s test_loss, test_acc = model5_1.evaluate(X_test_5, y_test_cat_5, verbose=0)
print(f'Độ chính xác trên tập Test: {test_acc*100:.2f}%')
print(f'Loss test: {test_loss:.4f}')
```

```
↗ Độ chính xác trên tập Test: 63.36%
Loss test: 1.0713
```

test_loss > 1.0: Mô hình có vấn đề (underfitting/overfitting).

Độ chính xác hơi thấp: 63.36%.

Thử nghiệm mô hình mới:

Mô hình gồm **4 lớp chính** (2 lớp tích chập + 2 lớp Dense):

1. Lớp Conv2D đầu tiên:

- 32 bộ lọc (filters) kích thước 4×4
- Kích hoạt bằng ReLU
- Input shape: $32 \times 32 \times 3$ (ảnh RGB 32×32 pixel)
- Output shape: $(None, 29, 29, 32)$

2. Lớp MaxPooling2D:

- Kích thước pool: 2×2
- Output shape: $(None, 14, 14, 32)$

3. Lớp Conv2D thứ hai:

- Tương tự lớp đầu nhưng **không cần chỉ định input_shape** (tự động kế thừa từ lớp trước)
- Output shape: $(None, 11, 11, 32)$
- Sau MaxPooling: $(None, 5, 5, 32)$

4. Lớp Dense (Fully Connected):

- Flatten(): Chuyển đổi tensor $5 \times 5 \times 32 \rightarrow 800$ giá trị 1D
- Dense(256): Lớp ẩn 256 nơ-ron + ReLU
- Dense(10): Lớp đầu ra 10 nơ-ron + Softmax (phân loại 10 lớp)

Độ chính xác : **65.48%** (Vẫn không cải thiện đáng kể so với model trên)



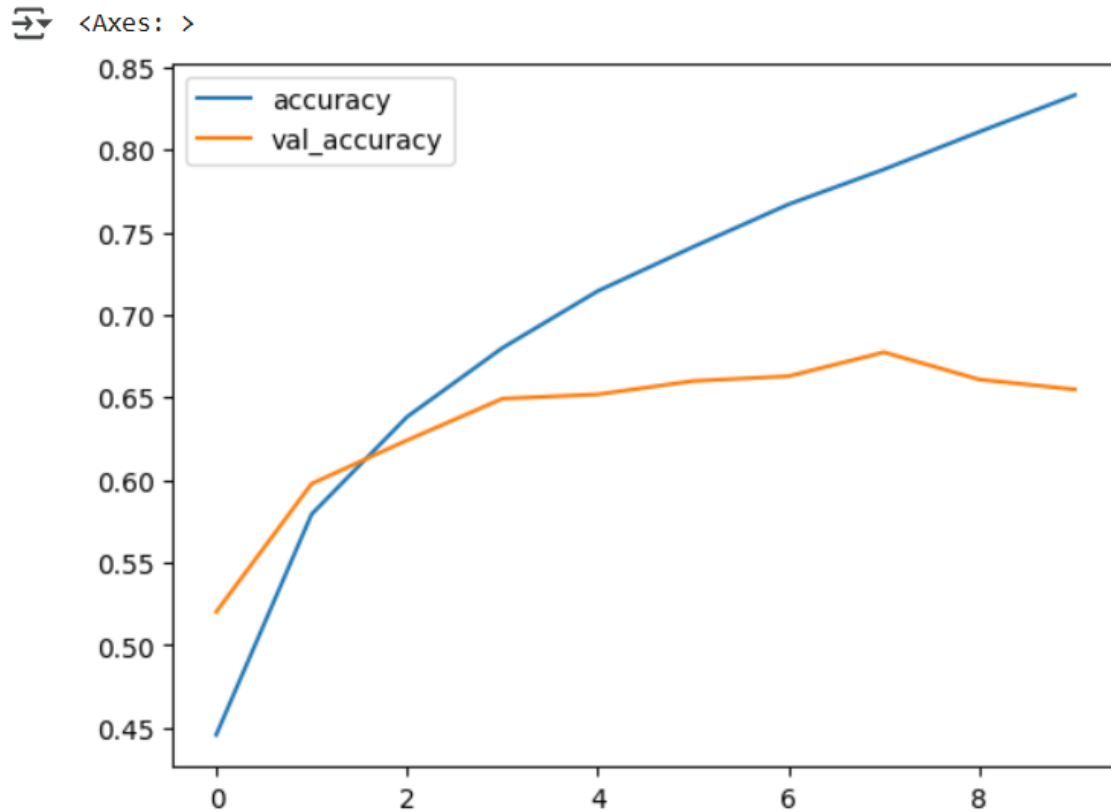
```
model.evaluate(x_test_5,y_test_cat_5,verbose=0)
```



```
[1.1723532676696777, 0.6547999978065491]
```

Xem xét biểu đồ:

```
[33] metrics[['accuracy', 'val_accuracy']].plot()
```



Overfitting

- **Training accuracy (accuracy)** tăng cao (~ 0.85) nhưng **validation accuracy (val_accuracy)** dừng ở $\sim 0.65-0.7$.
- Khoảng cách lớn giữa 2 đường cho thấy mô hình học quá khớp với tập train nhưng không tổng quát hóa tốt trên dữ liệu mới.